Project Proposal and Requirements Analysis Report

1. Team information

Name	Student number	Main responsibilities
Lennart Axel Conrad	2025080264	Chess-domain architecture, Java-to-C++ translation of chess and neural-network ideas from prior projects, C++ core implementation, tests, model/data validation, documentation integration.
Erik Mkrtchyan	2025080273	Raylib interaction workflows, visual layout and activation presentation, manual UI testing, tutorial/demo preparation, user-facing documentation review.

Both members participate in the final demo and must be able to explain the source code, build process, major classes, and testing evidence.

2. Project topic

The project is a native C++ chess neural-network visualizer. It combines a complete human-vs-human legal chess board with visualization panels for three neural-network architecture families:

- NNUE half-KP style accumulator and clipped output path.
- LC0-style convolutional residual network with board planes, trunk, policy, and WDL/value summaries.
- BT4-style token transformer visualization with token, attention, FFN, policy, and value views.

The topic fits the free-choice project requirement and is intentionally more advanced than a basic chess game because it includes move legality, notation, file I/O, model loading, tensor operations, tests, and real-time visualization.

3. Related prior work

The project uses the authors' prior chess programming experience as reference material, but the submitted runtime is C++:

- `https://github.com/LenniAConrad/chess-rtk` is a Java chess research toolkit with FEN/SAN handling, legal move generation, perft validation, engine analysis, dataset export, and chess rendering workflows.
- `https://github.com/LenniAConrad/chess-web` is a separate TypeScript web chess player/trainer with puzzle play, server-side validation, and browser UI experience.

The C++ submission does not call either repository at runtime. Ideas and some algorithms were manually translated or redesigned into C++17 classes, headers, tests, and raylib rendering code.

4. Requirements analysis

Requirement from course PDF	Project evidence
Classes	`Position`, `Game`, `MoveHistory`, `Tensor`, `ActivationSnapshot`, `App`, `BoardView`, network classes, loader classes, and view classes.
File I/O	`Config::load/save`, FEN save/load through `FenIo`, binary model reading through `WeightFileReader`, PGN export helpers, tutorial/demo evidence.
Multi-file `.h/.cpp` structure	`src/chess`, `src/game`, `src/nn`, `src/io`, and `src/viz` are split into headers and implementation files.
Inheritance and polymorphism	`INetwork` is the abstract base for NNUE/CNN/BT4 networks; `IActivationView` is the abstract base for visual panels.
Arrays and containers	Fixed arrays for board/bitboards/move list; STL vectors/maps/unordered maps for history, activation snapshots, config, and tensors.
Linked list	`MoveHistory` is a manually implemented doubly linked list for undo/redo and move-list rendering.
Templates	`Tensor<T, Rank>` implements a generic rank-aware tensor container.
C++ core implementation	Chess rules, NN forward paths, tensor ops, loaders, UI coordination, and tests are implemented in C++17.
User manual	`docs/user-manual.md`.
Design specification	`docs/design-spec.md` and `docs/class-diagram.png`.
Test cases	`docs/test-cases.md`, `tests/*.cpp`, and the tutorial GIF/video under `docs/`.
Summary report and AI usage	`docs/summary-report.md` and `docs/ai-usage.md`.

5. Functional requirements

1. The user can play a legal chess game with mouse input, legal target highlighting, promotion selection, undo/redo, and game-status reporting.
2. The user can load and save positions through FEN text files and can create custom legal positions in the setup editor.
3. The visualizer can switch between NNUE, CNN, BT4, and disabled activation views without changing the chess game state.
4. The app displays board overlays and per-architecture panels that explain how the current position moves through the selected neural-network structure.
5. The project can be built and tested from the command line with CMake scripts.

6. Non-functional requirements

- The chess core is separated from raylib so it can be unit tested without a graphics window.
- Runtime code should not depend on Java, TypeScript, Python, or external chess engines.
- Source files should be modular, with low coupling between chess rules, neural-network inference, file I/O, and visualization.
- Manual tests and the tutorial walkthrough must state input restrictions clearly so the final defense can reproduce the behavior.

7. Acceptance plan

Before final submission:

1. Run `./scripts/test.sh` and keep the result in `docs/test-cases.md`.
2. Build and launch the app from a clean checkout.
3. Record a tutorial demo showing play, all command buttons, FEN load/save, editor validation, architecture switching, detailed and abstract activation views, and the native search preview.
4. Export the required Markdown documents to PDF with `./scripts/export_docs.sh` if the course platform prefers PDF files.